Learning Augmentation Weights Through One Classifier Update in Corruption-Robust Continual Learning

Research draft
Author information to be supplied

Abstract

Can a continual learner improve robustness by scoring each sampled augmentation and learning its weight through a classifier update? We compare a chain-aware mixer with a recurrent MetaMix baseline on five sequential CIFAR-10 tasks. Both learn from clean cross-entropy and prediction consistency after one virtual classifier step. The chain-aware mixer scores individual training chains and uses independently weighted views to evaluate that step. Across three seeds at batch size 32, chain-aware mixing with class-aware InfoNCE reaches 37.49% corrupted class-incremental accuracy. MetaMix with the same classifier loss and original-image weight reaches 37.27%, and leads on corrupted task-incremental accuracy, 81.01% to 80.92%. Within MetaMix, fixing the original-image weight has a larger average effect on corrupted class-incremental accuracy under InfoNCE, although the direction varies by seed.

1. Introduction

An augmentation controller can reduce disagreement between a clean image and its transformed views by assigning more weight to the original. At the extreme, all views coincide. This presents a difficulty when prediction consistency is used to train the controller: a smaller loss can reflect an easier view even when the classifier has learned little about corruptions. In continual learning, the choice of views also affects the update shared by current examples and examples replayed from earlier tasks.

AugMix draws and blends short stochastic augmentation chains [3]. MetaMix adapts mixture weights using a recurrent summary of the learner and splits training operations into pseudo-seen and pseudo-unseen pools [4]. We study a modification with two linked choices. First, the mixer observes how each sampled seen chain changes the image's representation and prediction. Second, independent random weights determine the pseudo-unseen views on which the classifier update is scored. This design lets the mixer influence the outer loss only through its choice of training views.

Table 1 compares six configurations. It crosses classifier consistency with three mixing rules: chain-aware mixing at fixed strength, MetaMix at the same strength, and MetaMix with learned strength. The matched comparison asks whether the chain-aware design improves mean accuracy once both the classifier objective and the share of the original image are held fixed.

2. One-Step Mixing

Let θ denote classifier parameters and ϕ mixer parameters. At each batch we divide the augmentation operations into disjoint sets S and U. Both pools transform the same current and replay examples. For each image x, pool P and view $v \in \{1, 2\}$, the mixed input is

$$x_v^P = mx + (1 - m) \sum_{k=1}^{3} w_{vk}^P a_{vk}^P(x), \tag{1}$$

where a_{vk}^P is a sampled chain and the nonnegative chain weights sum to one. The coefficient m gives the original image's share of the blend.

What the controller observes. The MetaMix baseline uses an LSTM conditioned on current and replay features, replay loss and gradient alignment. For a given example it shares the three chain weights across views and pools. One variant also learns m and the other fixes $m = 0.5$. Resampling chains at fixed learner context leaves its weights unchanged.

The chain-aware controller fixes $m = 0.5$. For each seen chain it concatenates the change in pooled features with the change in class probabilities relative to the clean input. A shared scorer combines this detached descriptor with the recurrent state, then a softmax over the three scores supplies weights separately for each example and seen view. For each pseudo-unseen view we draw $w_v^U \sim \mathrm{Dirichlet}(1, 1, 1)$ independently of the controller and keep the draw fixed throughout the batch's mixer updates.

What the classifier learns. The support loss L_S combines current clean cross-entropy, replay cross-entropy with weight

Mixer	Classifier consistency	m	Clean class-IL	Corrupted class-IL	Clean task-IL	Corrupted task-IL
Chain-aware	JS	0.5	44.59	36.56	86.26	79.14
Chain-aware	InfoNCE	0.5	**46.26**	**37.49**	**87.12**	80.92
MetaMix	JS	0.5	43.51	36.38	86.89	80.05
MetaMix	InfoNCE	0.5	45.71	37.27	**87.12**	**81.01**
MetaMix	JS	Learned	46.16	36.43	86.65	79.83
MetaMix	InfoNCE	Learned	45.48	35.26	86.51	79.14

Table 1. Final accuracy (%) on Split-CIFAR-10 and CIFAR-10-C, batch size 32. Each seed (17, 23, 43) receives equal weight. Bold marks the highest displayed mean in each column, including the clean task-IL tie at two decimal places. All six configurations use clean cross-entropy plus JS for the mixer objective.

0.5, and mean squared error against stored replay logits with weight 0.2, following DER++ [1]. An independent replay sample supplies the logit targets. Classifier consistency has coefficient one. JS averages the KL divergences from the clean and two seen-view prediction distributions to their mean. Class-aware InfoNCE uses normalized pooled features at temperature 0.1. For each anchor, the two other views of the same batch entry are positives. Only different-class entries contribute negatives.

How the outer loss reaches the mixer. With classifier learning rate α, each mixer update evaluates

$$\theta^+(\phi) = \theta - \alpha \nabla_\theta L_S(\theta, \phi), \tag{2}$$

$$J(\phi) = L_U(\theta^+(\phi), \phi). \tag{3}$$

The outer loss L_U adds clean cross-entropy on the current and replay examples to JS on their clean and two pseudo-unseen predictions. Holding the sampled batch, incoming state and outer BatchNorm buffers fixed, the chain rule gives

$$\nabla_\phi J = (D_\phi \theta^+)^\top \nabla_{\theta^+} L_U + \partial_\phi L_U. \tag{4}$$

The partial derivative holds the virtual classifier fixed. For chain-aware mixing the outer images are independent of ϕ, so this term vanishes. For MetaMix the pseudo-unseen images depend on learned chain weights even when m is fixed, so a direct JS gradient can remain. Clean cross-entropy reaches either mixer through the classifier update. At $m = 1$ the three inputs coincide and their JS is zero under deterministic evaluation. This identity describes the consistency term alone.

Three mixer updates recompute the virtual classifier from the same starting parameters. The classifier commits the step computed with the incoming mixer once per batch. This is the full derivative of one SGD step. The MetaMix baseline uses the project's interpretation of the published solver.

3. Experimental Protocol

The 288,746-parameter SmallCNN learns five successive CIFAR-10 class pairs, $(0, 1)$ through $(8, 9)$, with ten epochs per task and a 500-example reservoir. Each batch has up to 32 current examples and up to 32 labeled replay examples. The classifier and mixer use constant SGD learning rates of 0.003 and 0.005. Each run processes 500,000 current-image presentations in 15,650 classifier updates.

Final evaluation uses 10,000 clean test images and 15 CIFAR-10-C corruptions at five severities [2]. Each corruption and severity receives equal weight. Class-IL predicts among all ten outputs. Task-IL restricts prediction to the true class pair. The pseudo-unseen operations used in training are sampled training transformations. CIFAR-10-C supplies only the test measurements.

Aggregation. For every configuration and metric, we take the arithmetic mean of the final accuracies from the three per-seed evaluation files and round it to two decimal places. We calculate percentage-point differences from the displayed means.

4. Results

Does candidate-aware mixing help at fixed strength? With $m = 0.5$ and classifier InfoNCE, chain-aware mixing has the highest observed clean and corrupted class-IL means (46.26% and 37.49%). Its margins over MetaMix are 0.55 and 0.22 percentage points, respectively. Both reach 87.12% clean task-IL at displayed precision. On corrupted task-IL, MetaMix leads by 0.09 points. With classifier JS the corrupted class-IL margin for chain-aware mixing is also small (0.18 points). Each comparison changes both the scoring of seen chains and the weighting of pseudo-unseen views.

Does classifier consistency matter? At fixed m, switching from JS to class-aware InfoNCE improves all four mean accuracies for both controllers. On corrupted class-IL the increases are 0.93 points for chain-aware mixing and 0.89 for MetaMix. With learned m, the same switch reduces MetaMix corrupted class-IL by 1.17 points. The effect of the classifier loss therefore depends on how MetaMix allocates weight to the original image in these runs.

Does the original-image weight explain a larger difference? Within MetaMix with classifier InfoNCE, fixing $m = 0.5$ increases mean corrupted class-IL by 2.01 points and corrupted task-IL by 1.87 points. The class-IL comparison varies across seeds (Table 2). With JS the corrupted class-IL means at fixed and learned m are close, 36.38% and 36.43%.

Original share	Seed 17	Seed 23	Seed 43
Learned m	39.08	35.70	31.01
Fixed $m = 0.5$	41.32	40.50	29.98
Fixed minus learned	+2.24	+4.80	−1.03

Table 2. Corrupted class-IL accuracy (%) and within-seed difference (percentage points). Fixing m improves two of the three paired runs.

What weights did the mixers learn? In the seed-17 MetaMix JS run, mean m rose from 0.5050 in the first epoch to 0.9000 in the last. The three chains together then contributed 10% of the average blend. The final-epoch mean largest chain weight was 33.46% for this baseline and 34.39% for chain-aware InfoNCE, close to the equal-weight value of $1/3$. These are diagnostics from individual runs. They do not measure the accuracy contribution of any particular chain.

5. Limitations and Conclusion

The three-seed means favor chain-aware InfoNCE for class-IL and fixed-strength MetaMix InfoNCE for corrupted task-IL. The small class-IL margins and the seed-dependent response to fixing m limit claims about the ordering on future runs. The matched rows change two mechanisms together. To identify the contribution of the descriptors, a comparison would need to vary descriptor access while holding outer-view sampling and per-view weight sharing fixed.

Reproducibility. The project README at revision `2bcca60` contains the three-seed table, aggregation notes and paired scores. The same revision provides the one-step training loop in `reproduce/unrolled_mixing.py`, the replication script in `scripts/repeat_unrolled_bs32.py`, and the MetaMix InfoNCE job in `jobs/unrolled_bs32/metamix_infonce.pbs`.

References

[1] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. In *Advances in Neural Information Processing Systems*, 2020.

[2] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations*, 2019.

[3] Dan Hendrycks, Norman Mu, Ekin D. Cubuk, Barret Zoph, Justin Gilmer, and Balaji Lakshminarayanan. AugMix: A simple data processing method to improve robustness and uncertainty. In *International Conference on Learning Representations*, 2020.

[4] Zhenyi Wang, Li Shen, Donglin Zhan, Qiuling Suo, Yanjun Zhu, Tiehang Duan, and Mingchen Gao. MetaMix: Towards corruption-robust continual learning with temporally self-adaptive data transformation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24521–24531, 2023.